

CS-860: ARTIFICIAL **INTELLIGENCE**

- **Uninformed Search**
 - **Search Problems**
 - **State Space**
 - **State Space Graphs**
 - **Search Trees**
 - **DFS/BFS**

**Text : Artificial Intelligence a Modern Approach
by Stuart Russel and Peter Norvig**

INSTRUCTOR:

DR. KUNWAR FARAZ AHMED KHAN

Uninformed Search

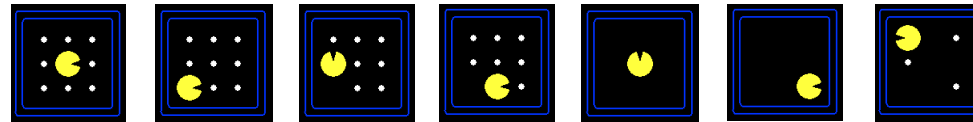


• Search Problems

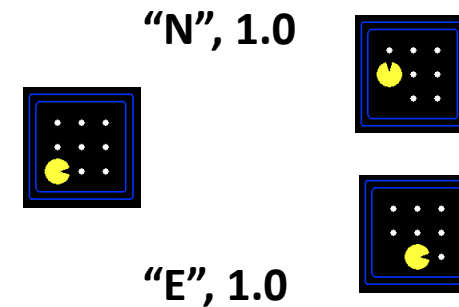
Search Problems

- A **search problem** consists of:

- A state space



- A successor function
(with actions, costs)



- A start state and a goal test

- A **solution** is a sequence of actions (a plan) which transforms the start state to a goal state



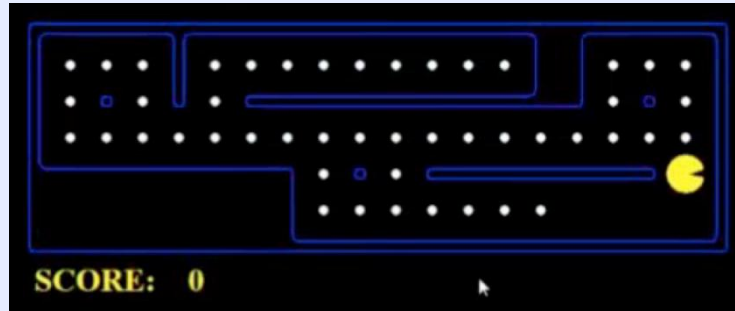
Uninformed Search



- Search Problems
- **State Space**
 - What's in a State Space

What's in a State Space?

The **world state** includes every last detail of the environment



A **search state** keeps only the details needed for planning (abstraction)

- **Problem: Pathing**
 - States: (x,y) location
 - Actions: NSEW
 - Successor: update location only
 - Goal test: is (x,y)=END
- **Problem: Eat-All-Dots**
 - States: {(x,y), dot booleans}
 - Actions: NSEW
 - Successor: update location and possibly a dot boolean
 - Goal test: dots all false



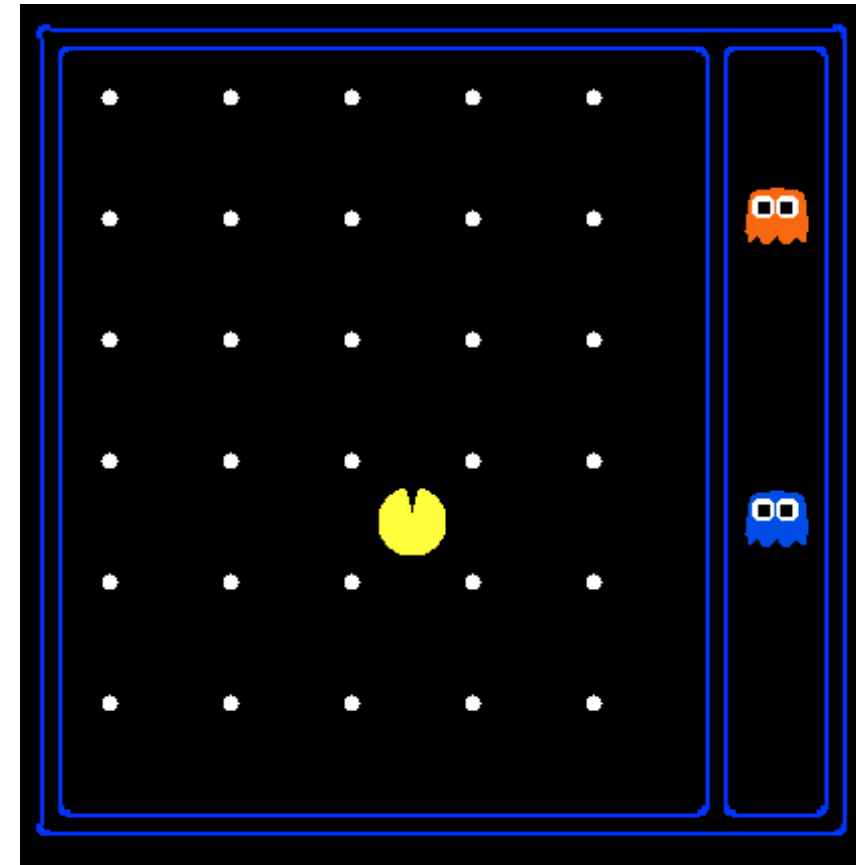
Uninformed Search



- Search Problems
- **State Space**
 - What's in a State Space
 - **State Space Size**

State Space Sizes

- World state:
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing: NSEW
- How many
 - World states?
 $120 \times (2^{30}) \times (12^2) \times 4$
 - States for pathing?
120
 - States for eat-all-dots?
 $120 \times (2^{30})$

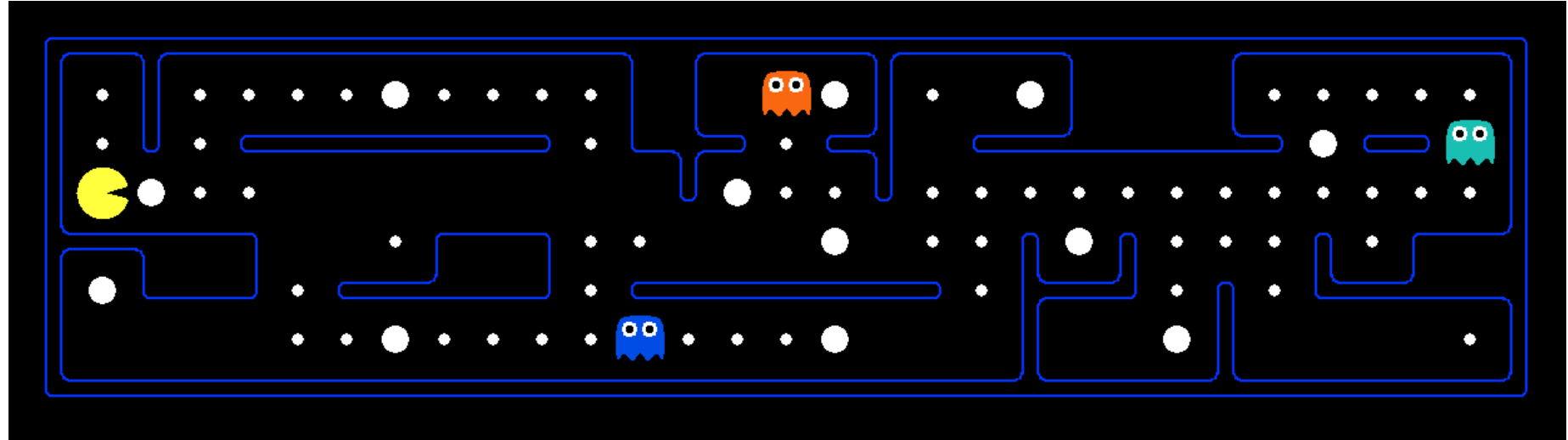


Uninformed Search



- Search Problems
- **State Space**
 - What's in a State Space
 - State Space Size
 - **Safe Passage**

Safe Passage



- Problem: eat all dots while keeping the ghosts perma-scared
- What does the state space have to specify?
 - (agent position, dot booleans, power pellet booleans, remaining scared time)



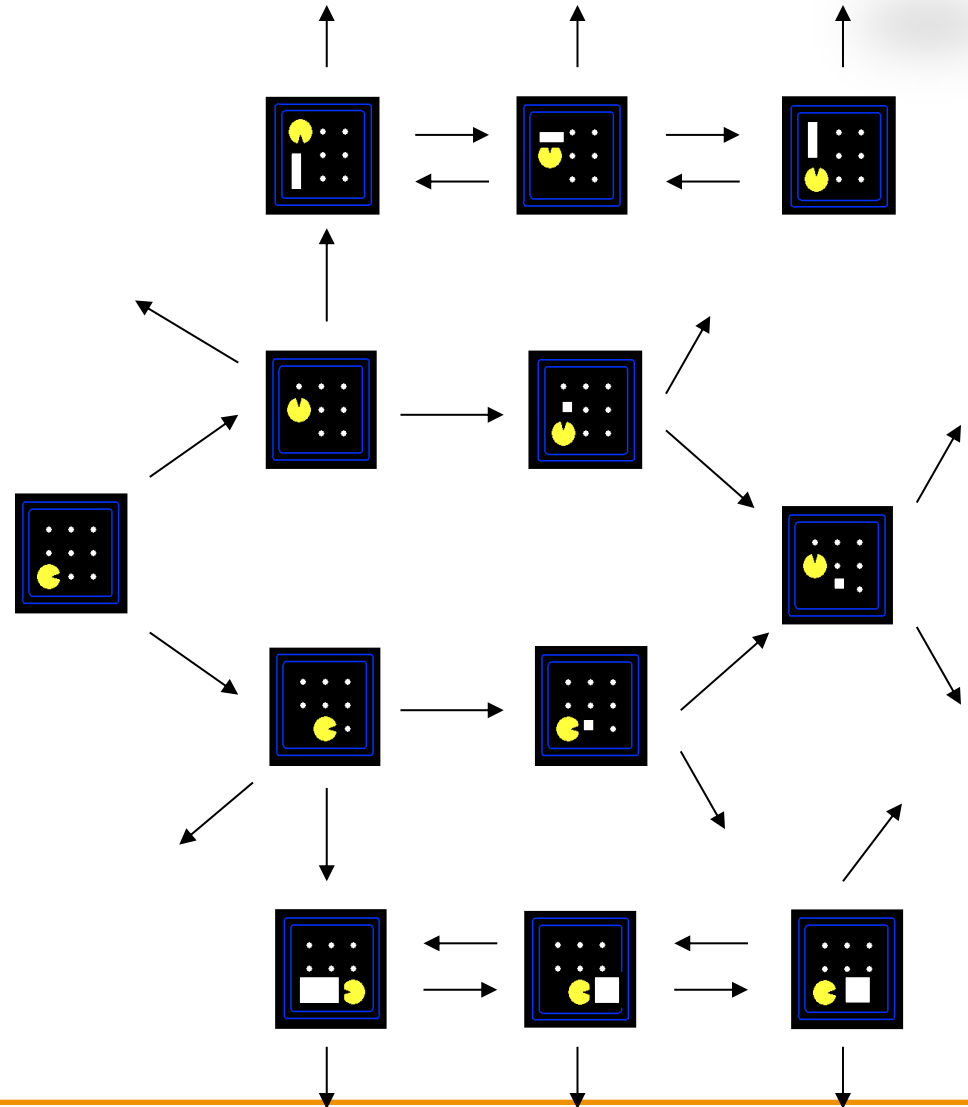
Uninformed Search



- Search Problems
- State Space
- **State Space Graphs**

State Space Graphs

- State space graph: A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



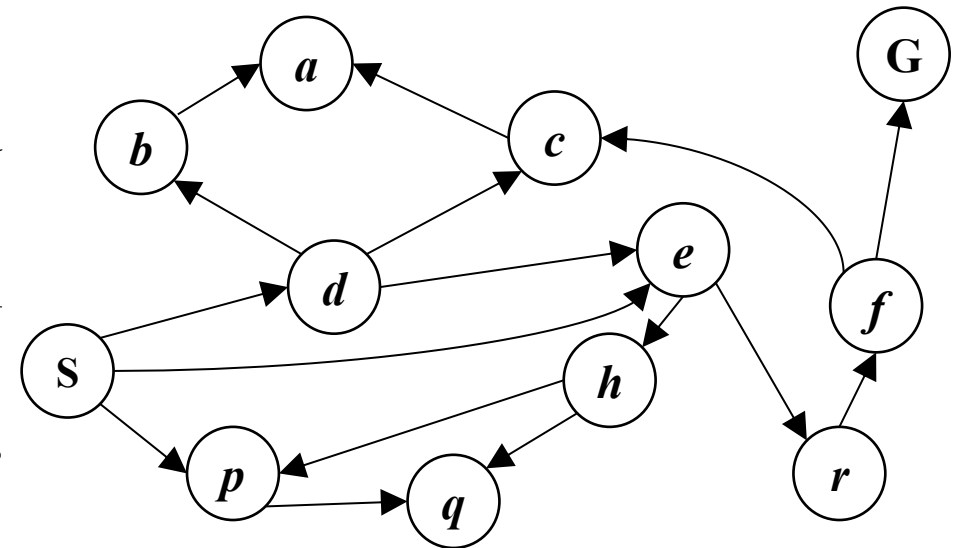
Uninformed Search



- Search Problems
- State Space
- **State Space Graphs**

State Space Graphs

- State space graph: A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



Tiny state space graph for a tiny search problem

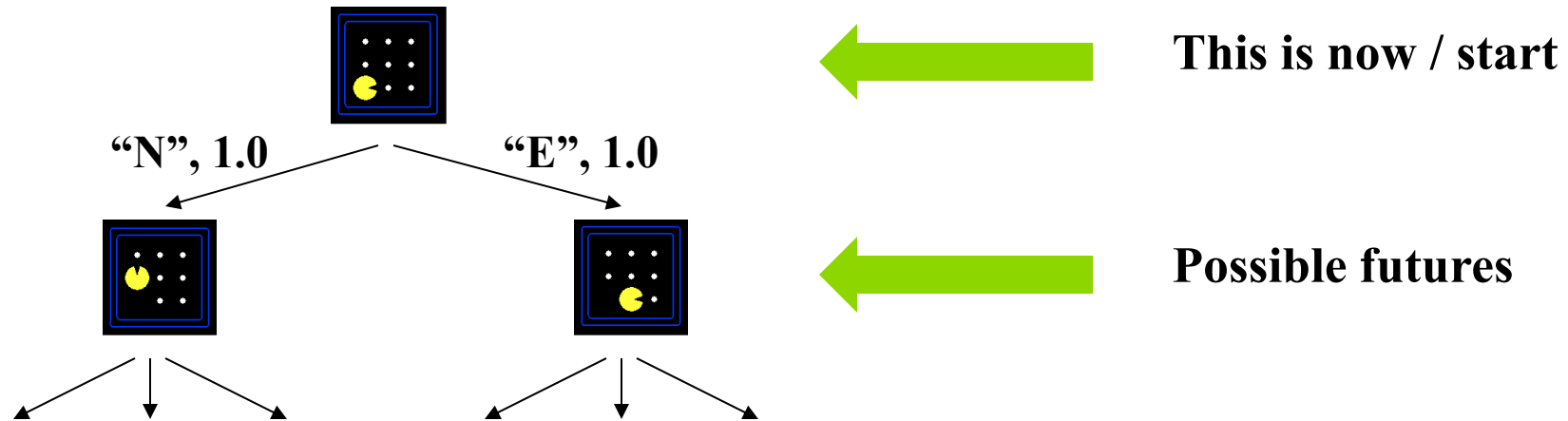


Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- **Search Trees**

Search Trees



- A search tree:
 - A “what if” tree of plans and their outcomes
 - The start state is the root node
 - Children correspond to successors
 - Nodes show states, but correspond to PLANS that achieve those states
 - **For most problems, we can never actually build the whole tree**



Uninformed Search

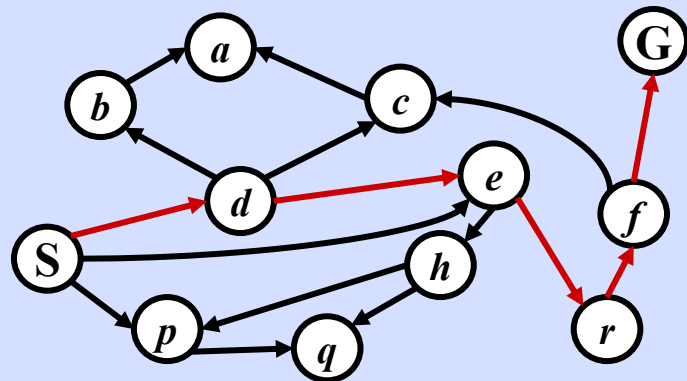


- Search Problems
- State Space
- State Space Graphs
- **Search Trees**

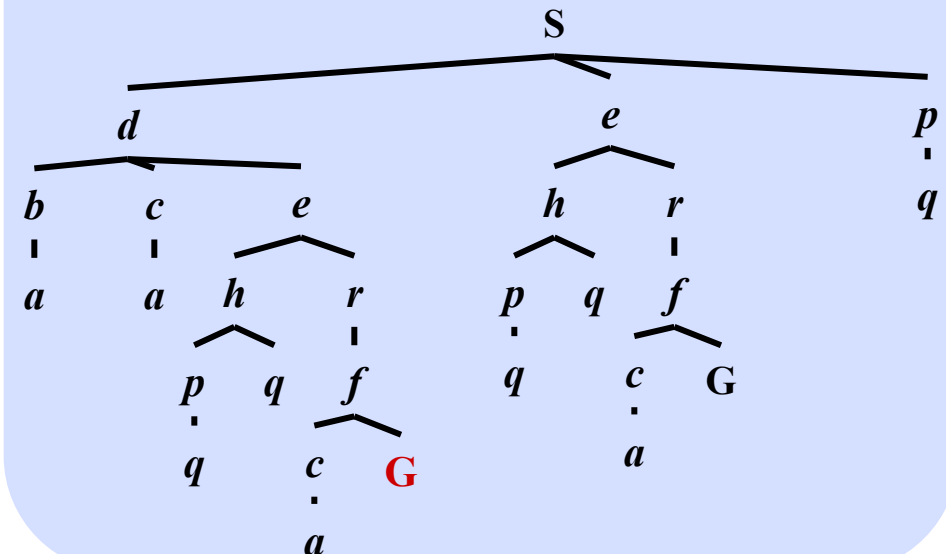
State Space Graphs vs Search Trees

- Each NODE in the search tree is an entire PATH in the state space graph.
- We construct both on demand – and we construct as little as possible.

State Space Graph



Search Tree



Uninformed Search

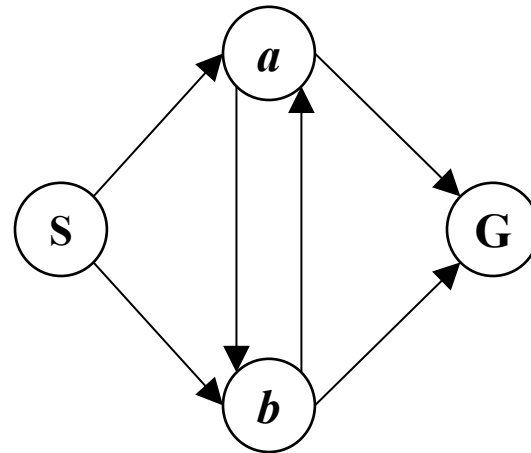


- Search Problems
- State Space
- State Space Graphs
- **Search Trees**

State Space Graphs vs Search Trees

Consider this 4-state graph:

How big is its search tree (from S)?



Uninformed Search

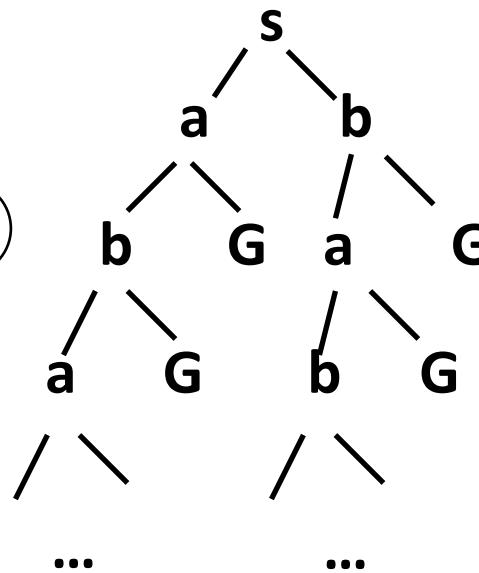
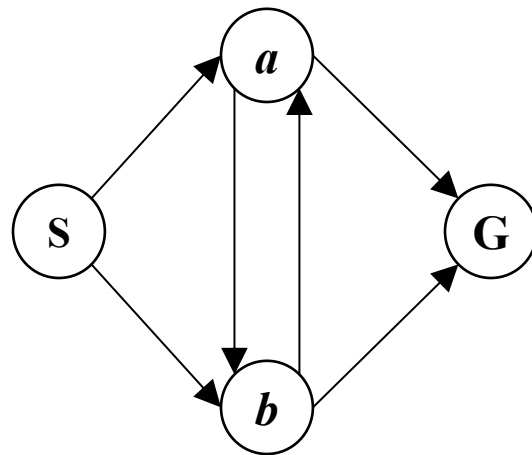


- Search Problems
- State Space
- State Space Graphs
- **Search Trees**

State Space Graphs vs Search Trees

Consider this 4-state graph:

How big is its search tree (from S)?



Important: Lots of repeated structure in the search tree!

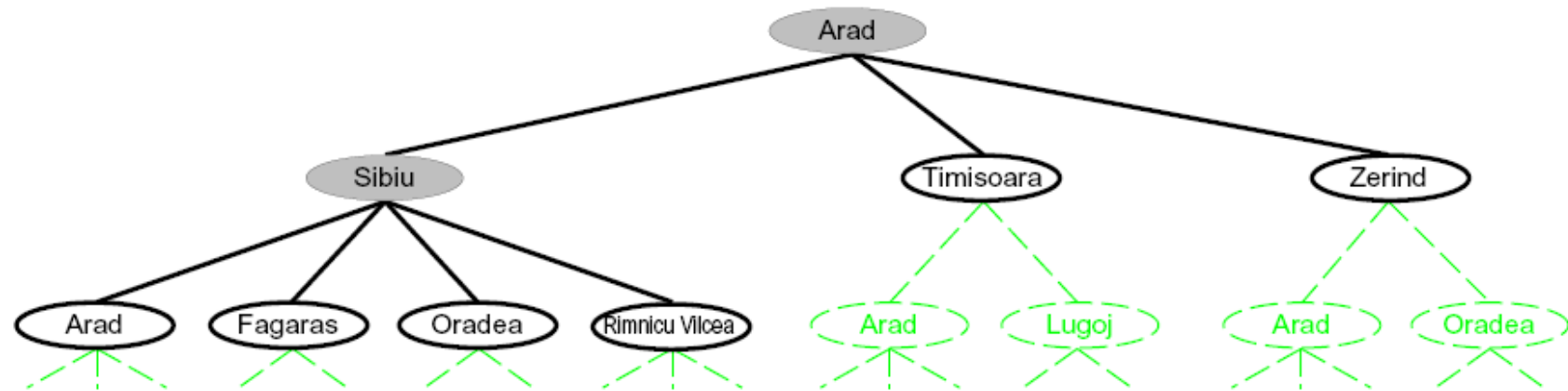


Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- **Search Trees**

Searching with Search Trees



- Search:
 - Expand out potential plans (tree nodes)
 - Maintain a **fringe** of partial plans under consideration
 - Try to expand as few tree nodes as possible



Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- **Search Trees**
 - **General Search Tree**

General Search Tree

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

- Important ideas:
 - Fringe
 - Expansion
 - Exploration strategy
- Main question: which fringe nodes to explore?

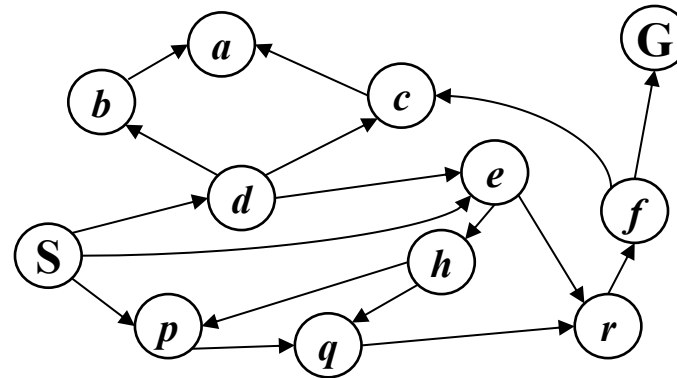


Uninformed Search

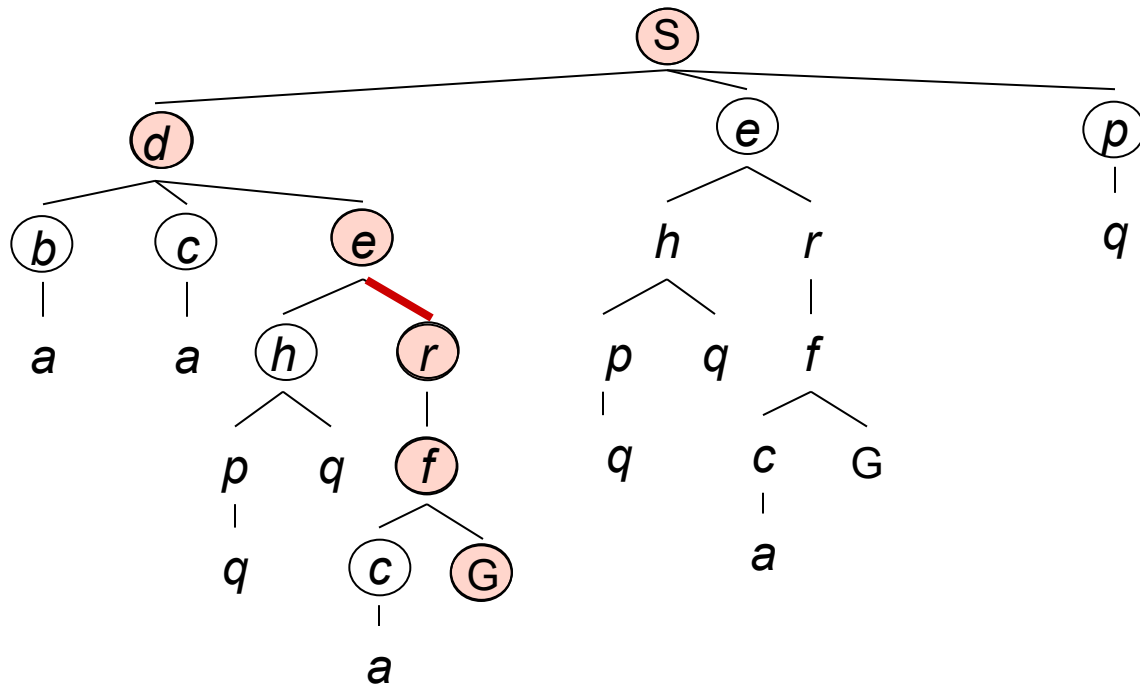
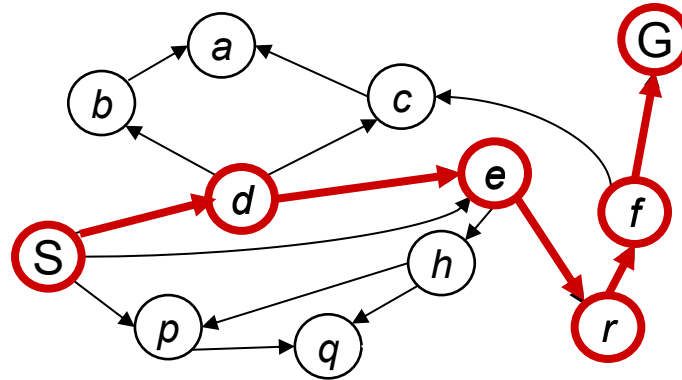


- Search Problems
- State Space
- State Space Graphs
- **Search Trees**
 - General Search Tree

General Search Tree - Example



Example: Tree Search



~~s~~
~~s → d~~
s → e
s → p
s → d → b
s → d → c
~~s → d → e~~
s → d → e → h
~~s → d → e → r~~
~~s → d → e → r → f~~
s → d → e → r → f → c
~~s → d → e → r → f → G~~

Uninformed Search

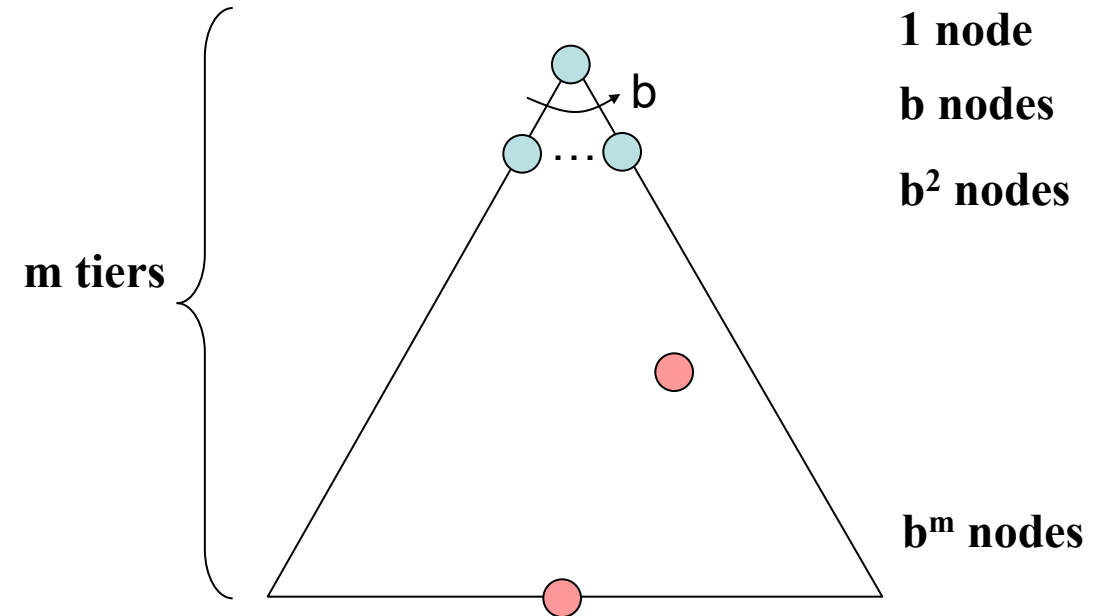


- Search Problems
- State Space
- State Space Graphs
- **Search Trees**
 - General Search Tree
 - **Search Algorithm Properties**

Search Algorithm Properties

- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?

- Cartoon of search tree:
 - b is the branching factor
 - m is the maximum depth
 - solutions at various depths



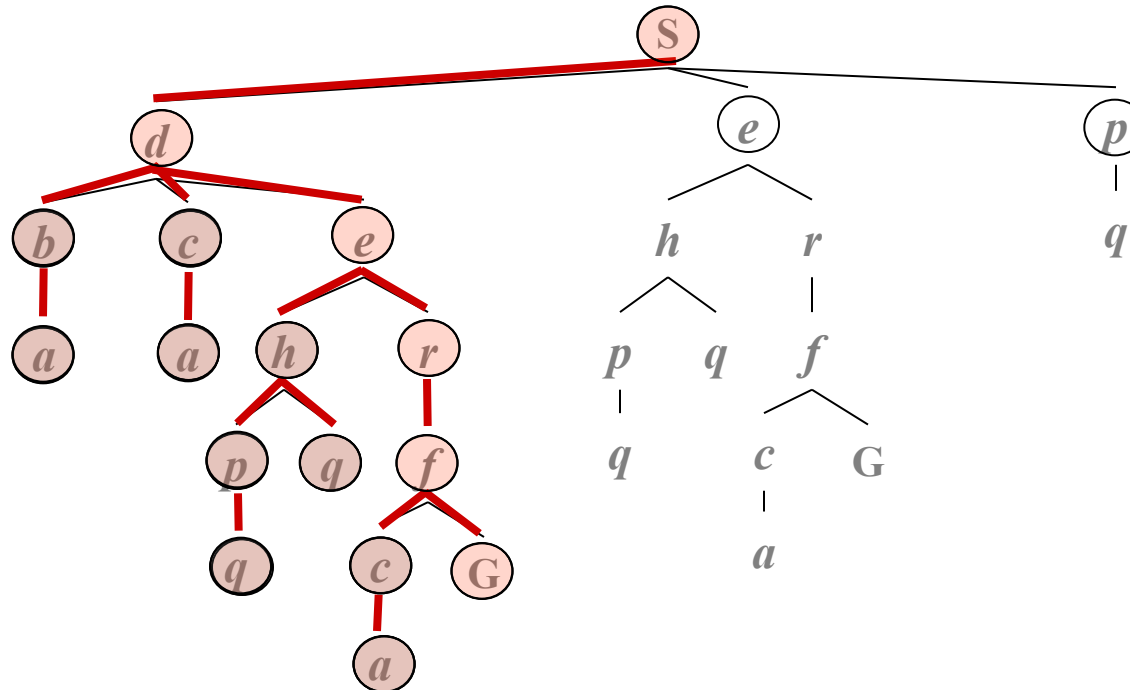
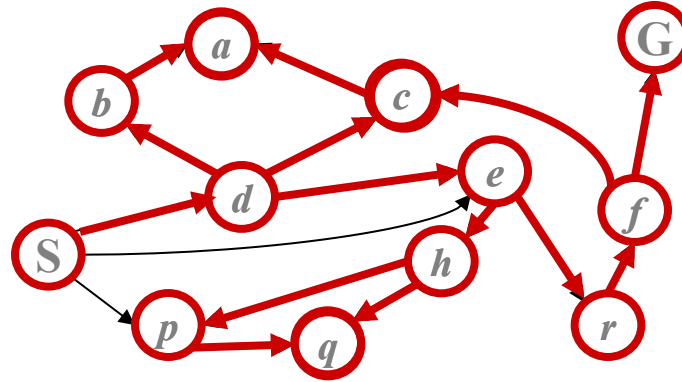
- Number of nodes in entire tree?
 - $1 + b + b^2 + \dots + b^m = O(b^m)$



Depth-First Search

Strategy: expand a deepest node first

Implementation:
Fringe is a LIFO stack



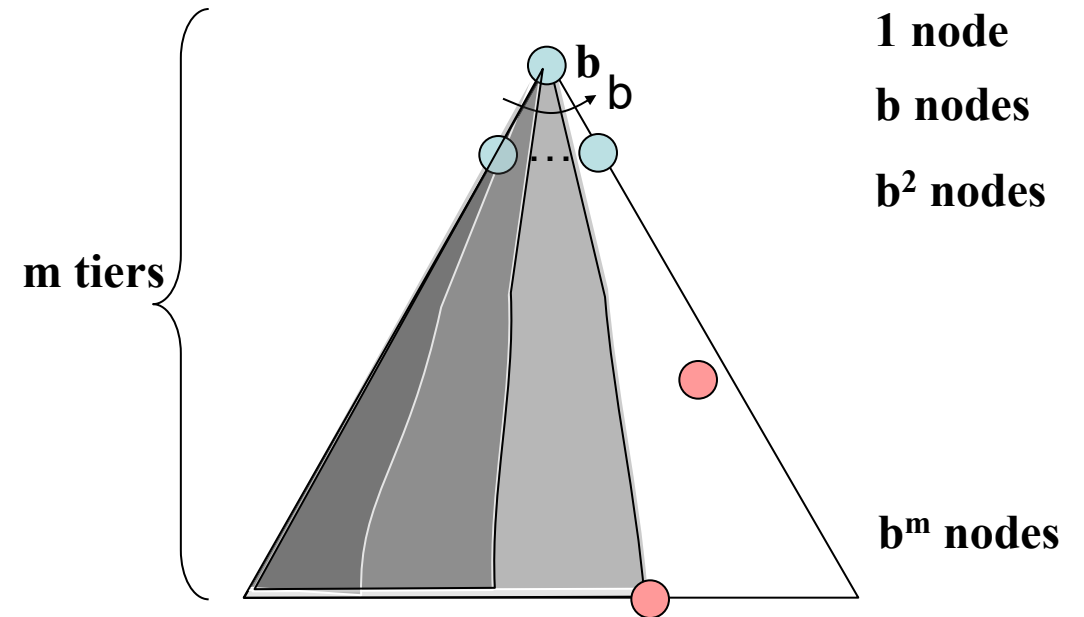
Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- Search Trees
- **Depth First Search (DFS)**
Properties

Depth First Search Properties

- What nodes DFS expand?
 - Some left prefix of the tree.
 - Could process the whole tree!
 - If m is finite, takes time $O(b^m)$
- How much space does the fringe take?
 - Only has siblings on path to root, so $O(bm)$
- Is it complete?
 - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
 - No, it finds the “*left most*” solution, regardless of depth or cost

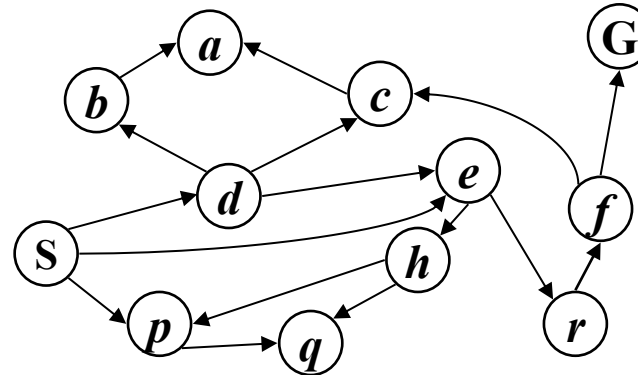


Uninformed Search



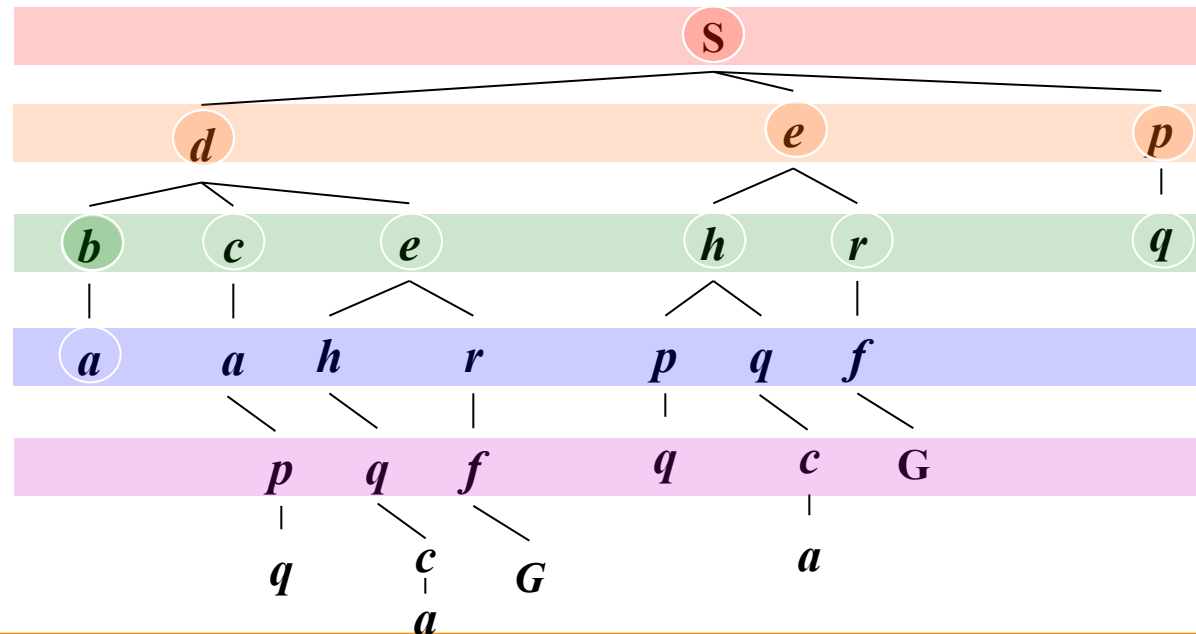
- Search Problems
- State Space
- State Space Graphs
- Search Trees
- Depth First Search (DFS) Properties
- **Breadth First Search (BFS) Properties**

Breadth First Search



- *Strategy: expand a shallowest node first*
- *Implementation: Fringe is a FIFO queue*

Search
Tiers



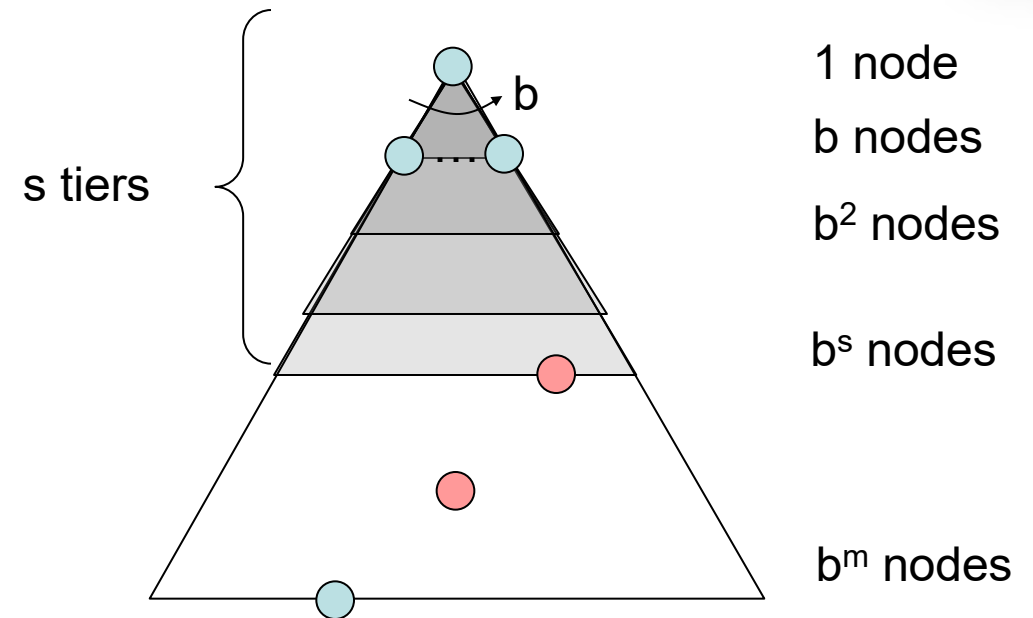
Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- Search Trees
- Depth First Search (DFS) Properties
- **Breadth First Search (BFS) Properties**

Breadth First Search Properties

- What nodes does BFS expand?
 - Processes all nodes above shallowest solution
 - Let depth of shallowest solution be s
 - Search takes time $O(b^s)$
- How much space does the fringe take?
 - Has roughly the last tier, so $O(b^s)$
- Is it complete?
 - s must be finite if a solution exists, so yes!
- Is it optimal?
 - Only if costs are all 1 (more on costs later)



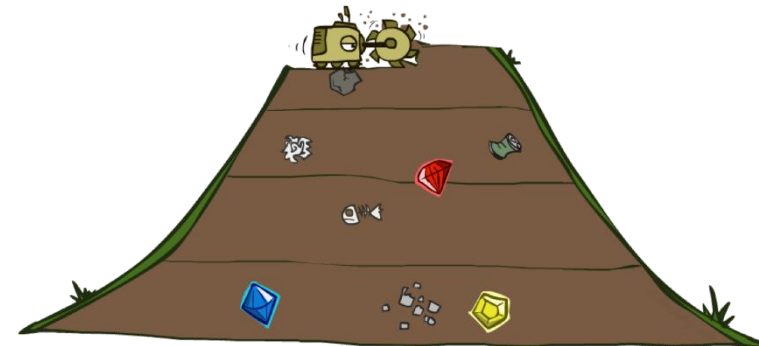
Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- Search Trees
- Depth First Search (DFS)
Properties
- Breadth First Search
(BFS) Properties
- **Comparison Questions**

Comparison Questions?

- When will BFS outperform DFS?
- When will DFS outperform BFS?

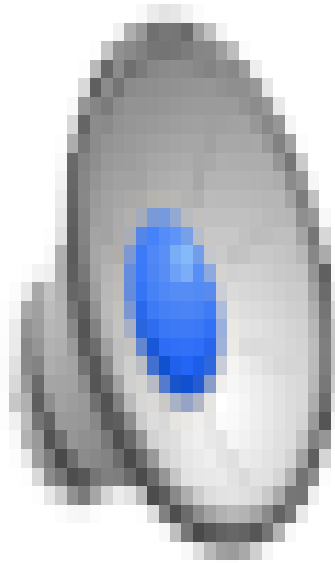


Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- Search Trees
- Depth First Search (DFS)
Properties
- Breadth First Search
(DFS) Properties
- **Comparison Questions**

Demo of Maze Water DFS / BFS (part I)

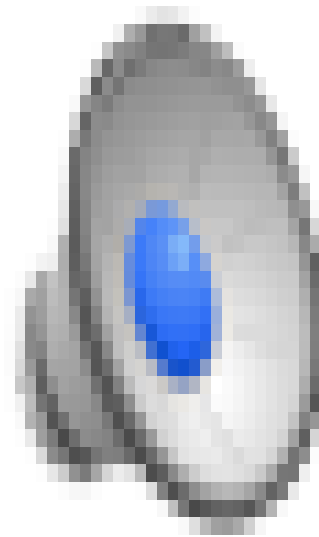


Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- Search Trees
- Depth First Search (DFS)
Properties
- Breadth First Search
(BFS) Properties
- **Comparison Questions**

Demo of Maze Water DFS / BFS (part I)



Uninformed Search



- Search Problems
- State Space
- State Space Graphs
- Search Trees
- Depth First Search (DFS) Properties
- Breadth First Search (BFS) Properties
- Comparison Questions
- **Iterative Deepening**

Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
 - Run a DFS with depth limit 1. If no solution...
 - Run a DFS with depth limit 2. If no solution...
 - Run a DFS with depth limit 3.
- Isn't that wastefully redundant?
 - Generally, most work happens in the lowest level searched, so not so bad!

